

TCP has multiple built-in **mechanisms** to ensure reliable and fair data transfer over a shared network:

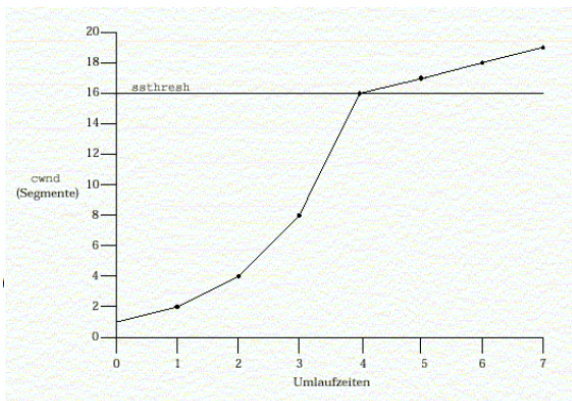
- **Handshake:** establishes the connection before data flows — ensures both ends are ready and agrees on initial sequence numbers.
- **Flow control:** receiver advertises how much buffer space it has — prevents the sender from overwhelming the receiver.
- **Congestion control:** sender adapts its sending rate based on network feedback — prevents the sender from overwhelming the *network*.

We focus on **congestion control** — how TCP detects and reacts to congestion, walk through a concrete scenario, and introduce a Markov chain model to capture TCP's behaviour analytically.

1 TCP Congestion Control

TCP congestion control regulates the sender's **congestion window** (cwnd) — the number of unacknowledged segments allowed in flight. It operates through three phases:

- **Slow Start:** exponential cwnd growth from 1 until ssthresh is reached.
- **Congestion Avoidance:** linear growth above ssthresh — careful probing of available bandwidth.
- **Congestion reaction:** two signals trigger a rollback — triple duplicate ACK (mild) or timeout (severe).



Start: cwnd = 1 segment.

Slow Start (exp.): cwnd $\times 2$ per RTT until ssthresh.

Congestion Avoidance (linear): cwnd +1 per RTT.

ssthresh init $\rightarrow$ user defined.

Triple dup ACK — receiver keeps ACKing same seq. no., but later pkts *are* arriving (out of order) $\Rightarrow$ one pkt missing, path still works. Fast Retransmit:

ssthresh = cwnd/2, restart at cwnd/2.

Timeout / RTO — RTO (est. from RTT) expires, no ACK at all $\Rightarrow$ neither pkts reaching dst nor ACKs returning, severe loss:

ssthresh = cwnd/2, cwnd = 1, restart slow start.

Throughput = $CWND \cdot MSS / RTT$
(for high-bandwidth links)

We can conclude: TCP maximises throughput while avoiding congestion via a dynamic congestion window.

1b Flow Control & Congestion Window Interaction & (interaction with Layer 2 (WLAN))

Flow control (rwnd) and congestion control (cwnd) are **independent** limits — both apply at all times, the sender is bounded by whichever is smaller:

Sliding window / rwnd $\rightarrow$ protect *receiver* buffer (set by receiver in each ACK)

Congestion window / cwnd $\rightarrow$ protect *network* (set by sender via AIMD)

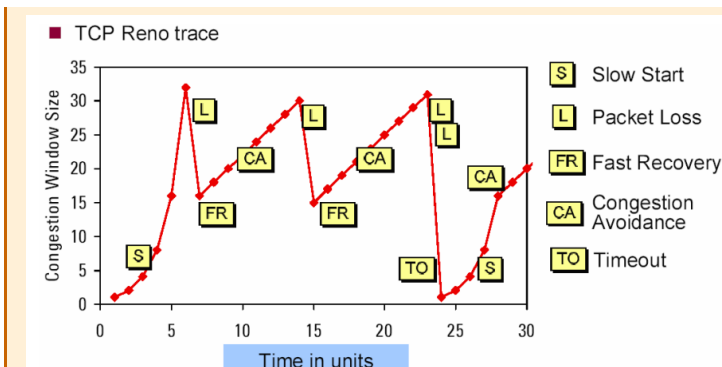
Usable window = min(rwnd, cwnd) — cwnd may grow past rwnd, but min caps actual sending

L2 interaction (e.g. WLAN): L2 protocols like 802.11 have their own **retransmissions** and **rate adaptation** (dropping from e.g. 54 $\rightarrow$ 11 Mbps on poor signal). Both happen invisibly below TCP.

TCP never sees a packet loss $\Rightarrow$ cwnd never backs off $\Rightarrow$ cwnd keeps growing, until it hits the effective L2 throughput ceiling.

This effect is clearly observable: TCP throughput plateaus at the L2-adapted rate rather than the nominal link rate.

Note and Example (Only if they ask)



TCP Reno trace:

- S=Slow Start
- L=Loss
- FR=Fast Recovery, FR only halves — much less disruptive.
- CA=Cong. Avoid
- TO=Timeout, TO resets cwnd= 1;

2 Fixpoint Model: TCP + Network Mutually Dependent

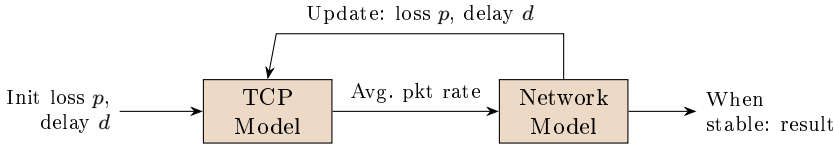
TCP and the network are **mutually dependent** — each affects the other continuously:

TCP sends faster $\Rightarrow$ network congests $\Rightarrow$ loss p and delay d increase $\Rightarrow$ TCP backs off $\Rightarrow$ congestion eases $\Rightarrow$ TCP speeds up again.

They **co-determine** each other — neither can be solved in isolation.

We model this with a **fixpoint model** (basically to get queue bottleneck/congestion):

- a feedback loop between a TCP model and a network model iterated until stable
- The diagram below is an interpretation of the expected interaction between the two.



TCP model (Padhye et al.): analytical formula capturing slow start, cwnd dynamics and backoff — takes loss p and delay d as input, outputs **throughput** T (pkts/s).

Network model (e.g. M/M/1/K): chosen to reflect expected network behaviour — buffer overflow $\Rightarrow$ congestion, packet drops. Takes T as arrival rate, returns updated p and d .

Iteration:

1. Initialise with a guess for p, d
2. TCP model $\rightarrow$ compute $T(p, d)$
3. Network model $\rightarrow$ compute $p(T), d(T)$
4. Repeat until p, d, T no longer change $\rightarrow$ **fixpoint reached**

Padhye [Sigcomm 98] (if asked) — T = throughput (pkts/s):

$$T \approx \frac{1}{RTT} \min \left(W_{\max}, \frac{1}{\sqrt{\frac{2bp}{3}} + \frac{T_0}{RTT} \min(1, 3\sqrt{\frac{3bp}{8}}) p(1+32p^2)} \right)$$

- b : number of packets acknowledged per ACK
- RTT : average round-trip time (incl. queuing delay)
- $W_{\max}$: maximum congestion window size
- T_0 : average timeout interval
- p : fraction of retransmitted packets (loss rate)

3 Results: File Transfer Delays — Model vs. Measurement

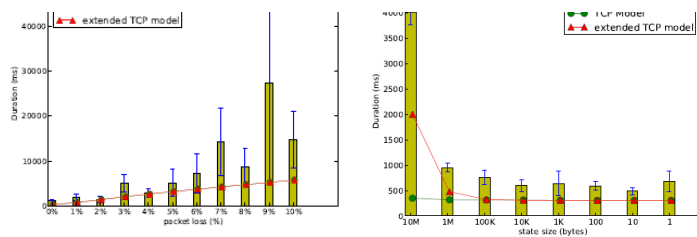


Fig. 9. Duration of a migration. Link packet loss is varied from 0 to 10%.

TCP Model from [23] N. Cardwell, S. Savage, and T. Anderson, "Modeling tcp latency," vol. 3,

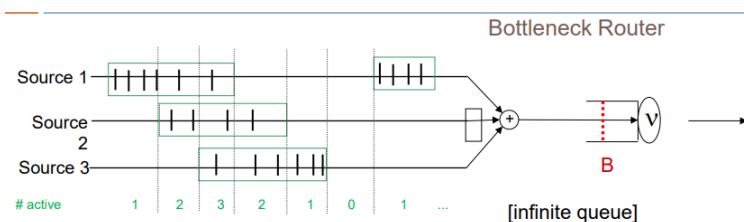
Model deviates significantly.

At 9% packet loss: TCP model $\approx$ 500 ms, measured $\approx$ 27000 ms — $50\times$ error.

Extended TCP model (adds bandwidth limits) tracks measured much better.

Use fixpoint as estimate only.

4 Model 3: Multiple Connections — Discouraged Arrivals



j connections share bottleneck (rate ν , buffer B).

Rate per conn: λ_p if $j\lambda_p \leq \nu$; else $\delta\nu/j$.

Why $\delta < 1$ (not 100% util.): TCP does slow start, backs off on every loss/timeout, and never runs at full rate — it constantly probes and retreats. So actual throughput $\approx \delta \times$ ideal, with $\delta \leq 1$ derived from TCP loss behaviour.

Congestion feedback $\rightarrow$ **Markov chain** (no closed form). Buffer fills before max TP is reached. Model 3 = ON/OFF arrivals with queue-dependent rate.

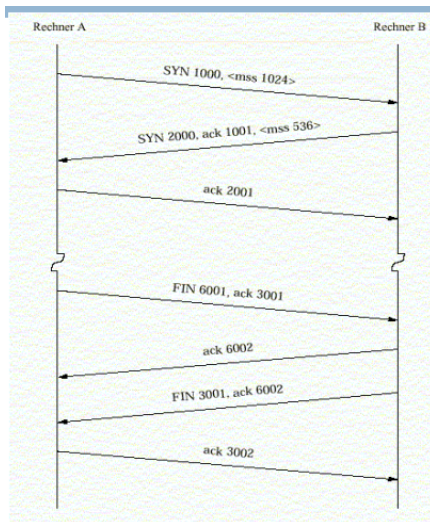
5 Connection Level Model — Why Analytically Better

What it does (Heyman et al.): N TCP sources share bottleneck rate ν . Modified Processor Sharing (M/M/1//N/PS): j active conns each get λ_p if $j\lambda_p \leq \nu$, else $\delta\nu/j$. Yields **closed-form** stationary distribution — no iteration.

vs. Model 3: ON/OFF arrivals throttled by congestion $\Rightarrow$ Markov chain, no closed form, numerical only.

Why better: direct analytical throughput distribution; captures connection arrival/departure dynamics; no heuristic fix-point iteration required.

6 TCP Handshake, Sequence Numbers & Teardown



3-Way Handshake:

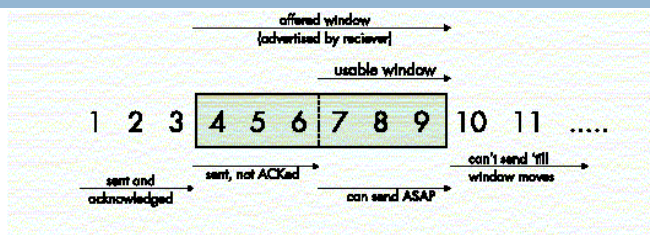
1. **SYN:** Client $\rightarrow$ Server; $\text{seq} = x$, MSS option
2. **SYN-ACK:** Server $\rightarrow$ Client; $\text{seq} = y$, $\text{ack} = x+1$, MSS option
3. **ACK:** Client $\rightarrow$ Server; $\text{ack} = y+1$

Seq numbers count *bytes* sent (not packets) — increments by bytes in each segment. **MSS** negotiated in SYN options; min of both sides is used.

FIN (graceful): 4-way close — FIN, ACK, FIN, ACK; each side closes independently (half-close). **RST:** immediate abort on error or connection refused.

7 Flow Control vs. Congestion Control

TCP Flow/Congestion Control



- **Sliding Window:** # packets sent and un-acknowledged limited by
 - ▣ receiver buffer (receiver advertised window)
 - ▣ congestion window (purpose: protect intermediate nodes)
- **Congestion window** dynamically adjusted („probing“ for usable bandwidth): increase gradually until congestion indicated via
 - ▣ Timeouts
 - ▣ Duplicate Acknowledgements

Flow Control (protects *receiver*):

Receiver advertises **rwnd** = offered window = free buffer space in ACKs. This sets the right edge of the window.

Sliding window: left edge advances as ACKs arrive; right edge = $\min(\text{cwnd}, \text{rwnd})$.

Congestion Control (protects *network*):

cwnd probes for usable bandwidth (slow start, CA). Also limits right edge.

Usable window = right-edge portion not yet sent (segments 7–9 in figure).